Mission 05: Solution Report

Contributors: Syed Hashir Sohail, Yasan Malik, Usman tariq qureshi, Sufyan Khalid
Group: sustkever (Group 17)
Course: Conversational AI, Winter Semester 2026/27
Prepared: 9 November 2026

1. Mission objective and scope

Mission 05 investigates source-grounded retrieval and response evaluation for AURA’s Dual-Path Associative Memory Core. Its four parts cover HNSW vector indexing, BM25 and Reciprocal Rank Fusion (RRF), an empirical NotebookLM grounding audit, and prompt injection detection with RAG Triad metrics.

This report describes the submitted code, the notebook’s retained outputs, and the saved evidence. It covers the entire mission, including incomplete requirements. It does not claim that a complete production GraphRAG system or an integrated end-to-end answer generator has been built. Contributions are credited collectively to the four contributors above; individual task ownership and contribution percentages have not been established.

The local verification for this report reran the NotebookLM CSV consistency check and the Part 4 demonstration. The NumPy-based HNSW and full hybrid benchmarks were not rerun because NumPy is absent from the available Python environment. Recorded results are identified separately below.

2. Files and overall design

The mission contains these principal deliverables:

- 01_HNSW_Vector_Index.py: pure Python/NumPy approximate vector search, graph construction, self-checks, and benchmarking.
- 02_BM25_and_RRF_Combiner.py: full lexical retrieval, dense HNSW retrieval, rank fusion, and evaluation. The separate _starter.py file retains a simpler implementation.
- 03_NotebookLM_Grounding_Lab_template.md: completed lab narrative, recorded responses, citation evidence, refusal probes, audio transcript, and evaluation.
- 03_NotebookLM_RAG_Audit.py: standard-library aggregation of citation and probe CSVs, with a consistency check against recorded lab totals.
- notebooklm_audit_report.md: detailed Part 3 findings and limitations.
- 04_Prompt_Injection_and_RAG_Audit_starter.py: pattern-based sanitization and token-overlap evaluation demonstration.
- ps05_solution.ipynb: executed Part 1 walkthrough and recorded HNSW results. Parts 2 through 4 are not integrated into this notebook.
- hnsw_benchmark_results.json: saved 5,000-vector benchmark configuration and measurements.
- citation_audit_*.csv and refusal_probes.csv: badge-level, claim-level, and refusal evidence used by the audit script.

The proposed retrieval flow is: query -> BM25 and dense HNSW rankings -> RRF -> retrieved context -> injection scan -> generated answer -> grounding audit. The repository supplies components and demonstrations for this flow; it does not connect all stages into one deployed service. NotebookLM is audited separately as an external grounded system.

3. Part 1: HNSW vector index

Method

The implementation normalizes vectors so cosine similarity becomes a dot product. Scalar and vectorized helpers support comparisons, including zero-vector handling. HNSW assigns nodes to exponentially sparser upper layers:

```
level = floor(-ln(U) * m_L)
m_L = 1 / ln(M)
P(level >= k) = M^(-k)
```

Insertion connects nodes through the hierarchy. Search starts at the upper entry point, follows greedy improvements, then uses a wider candidate search on layer 0. M controls neighborhood size; ef_construction and ef_search control construction and query effort. Increasing search effort usually improves recall at additional latency.

The self-checks cover cosine calculations, the layer distribution, agreement with exact search when the search width is large enough, degree limits, sorted candidate results, and deterministic construction for a fixed seed. The notebook retains successful output for these five check groups.

Saved JSON results

The current JSON uses 5,000 synthetic vectors, 768 dimensions, 200 queries, top-10 retrieval, M=16, ef_construction=100, ef_search=64, and seed 42. Its scaling measurements are:

Vectors	HNSW ms	Flat ms	Recall@10
1,000	1.047	0.101	1.0000
2,500	1.896	0.377	0.9955
5,000	2.232	0.713	0.9890

At 5,000 vectors, HNSW evaluates about 1,196 similarities per query, rather than all 5,000. However, it is slower than optimized flat search on this saved run: the reported speedup ratio is 0.320. Build time is approximately 26.1 seconds and layer sizes are 5,000 / 283 / 18 / 2.

The separate search-width sweep records:

ef_search	HNSW ms	Recall@10
16	0.863	0.8120
32	1.375	0.9325
64	2.299	0.9890
128	4.247	0.9985
256	5.425	1.0000

Notebook evidence and inconsistency

The notebook retains an earlier configuration with 25,000 vectors. At that size it records HNSW latency of 1.567 ms, flat latency of 3.018 ms, recall@10 of 0.945, and a speedup of 1.93. These are retained notebook outputs, not the current JSON’s results.

The notebook normally reads the JSON without rerunning the experiment. Executing it now with RERUN = False would display the current 5,000-vector results instead. The report therefore treats the two artifacts as separate recorded runs. Neither supports a blanket claim of sub-millisecond search or a universal speed advantage. Dataset structure, search width, implementation overhead, and hardware affect the outcome; these finite measurements do not prove asymptotic complexity.

4. Part 2: BM25 and hybrid rank fusion

The full implementation tokenizes a small starship-document corpus and calculates BM25 with k1=1.5 and b=0.75. Term frequency saturates, inverse document frequency emphasizes rarer terms, and document-length normalization prevents long documents from gaining an automatic advantage. Zero-score documents are filtered from the full implementation’s ranked lexical results.

The dense path uses ConceptEmbedder, a deterministic, hand-designed concept-cluster embedding system, and the Part 1 HNSW module. It demonstrates semantic grouping without downloading a pretrained sentence encoder. Its results should be interpreted as a controlled educational retrieval experiment.

Rank fusion combines document positions rather than incompatible raw sparse and dense scores:

```
RRF(document) = sum(1 / (60 + rank_in_each_list))
```

Ranks start at one. A document appearing in both lists receives both contributions. The module also exposes labeled ranking provenance and term-level BM25 breakdowns for inspection.

Evaluation separates exact-code and paraphrase queries, then reports combined performance. Implemented metrics include mean reciprocal rank, NDCG@10, precision@10, and recall@10. A smoothing sweep compares RRF k values of 1, 10, 30, 60, 100, and 200. Self-checks cover BM25 behavior, rank fusion arithmetic, ranking metrics, and concept embeddings.

The intended behavior is for sparse retrieval to preserve serial codes and dense retrieval to help with paraphrases. No standalone saved full-hybrid benchmark output was present in the reviewed files, and that benchmark was not rerun locally. This report therefore does not invent hybrid performance numbers or claim that RRF outperforms every component on the available queries.

5. Part 3: NotebookLM grounding audit

Corpus and procedure

The recorded audit, dated 7 November 2026, used Document A, Lewis et al.’s RAG paper, and Document B, Edge et al.’s GraphRAG paper. Document C, the USSE Noosphere Technical Archive Extract, was unavailable according to the lab. The results cover only the first two documents.

Five cross-document queries were evaluated. Each opened citation badge received a verdict: supported, partial, or unsupported. Strict citation precision counts only supported badges; lenient precision also counts partial badges. Claim-level evaluation asks whether multiple cited passages jointly support an assertion. These measure different things and must be reported separately.

Citation results

Query	Badges	Supported	Partial	Unsupported	Strict
Q1	84	54	29	1	64.3%
Q2	157	58	98	1	36.9%
Q3	12	8	4	0	66.7%
Q4	0	0	0	0	n/a
Q5	9	3	6	0	33.3%
Total	262	123	137	2	46.9%

Overall lenient precision is 99.2%. Q4 has no citation badges, so its citation precision is undefined. Zero badges must not be interpreted as perfect grounding. The claim-level support rates among cited assertions are 53/64 for Q2 (82.8%), 7/8 for Q3 (87.5%), and 4/5 for Q5 (80.0%).

The central finding is that a relevant citation often omits the exact number or mechanism claimed. Counting citation presence alone would overstate grounding. Dense lists of technical details can require several passages; the audit also records wording that goes beyond the cited evidence.

Refusal and audio results

All three recorded negative-constraint probes received refusals: an out-of-domain fictional temperature, an unreported GraphRAG MMLU score, and a cross-paper MS-MARCO result. This is a three-probe sample from one session. P2 included ten badges in added context, with strict precision of 20.0%. P3’s volunteered context included an incorrect 42.8 ROUGE-L figure where the lab records 40.8 as the paper’s value. A correct refusal does not guarantee that surrounding text is accurate.

The recorded Audio Overview lasts 4 minutes 26 seconds; its supplied transcript has 767 words. Factual fidelity, persona differentiation, and pedagogical value each receive 4/5. Fourteen claims were categorized as eight supported, one supported with one word added, two overstated, one misleading, one absent from the sources, and one speculative.

Audio assessment used transcript text. Delivery, pauses, and interruptions were not assessed, and the transcript exceeds the required two-minute excerpt. The lab credits Claude with assigning citation verdicts and performing a second pass. The contributors’ independent spot-check remains pending; it must not be presented as completed.

Local verification

Running 03_NotebookLM_RAG_Audit.py --check succeeded while preparing this report. It recomputed the CSV totals and matched the script’s recorded Section 2.7 totals. This verifies consistency of the stored audit counts, not the correctness of every verdict or a new live NotebookLM session.

6. Part 4: Injection sanitization and RAG Triad

PromptInjectionSanitizer compiles seven regular-expression patterns for directives such as ignoring previous instructions, overriding a system prompt, entering developer mode, and revealing internal instructions. Matching substrings are replaced with [QUARANTINED_ADVERSARIAL_PAYLOAD]; a boolean reports detection.

The supplied example is flagged successfully in the local demonstration. However, the replacement leaves surrounding text, including the instruction to output encryption keys. The routine does not remove the entire chunk. Its flag needs to be handled by a caller; a positive demonstration does not establish protection against paraphrases or arbitrary injection attacks.

The three implemented scores are token-set overlap heuristics:

```
context_relevance = |query_words intersect context_words| / |query_words|
faithfulness = |response_words intersect context_words| / |response_words|
answer_relevance = |query_words intersect response_words| / |query_words|
```

Denominators are clamped to at least one. Local demo results were context relevance 0.6667, faithfulness 1.0000, and answer relevance 0.6667. These scores do not establish semantic relevance or claim entailment. Shared words can occur in contradictory statements, and punctuation affects this simple whitespace tokenization.

Part 4 remains a starter. The reviewed implementation does not provide the briefing's embedding-based detector, dedicated hidden-image exfiltration detection, semantic claim evaluation, or structured per-turn provenance JSON logging. The expected 04_RAG_Provenance_and_Safety.py deliverable is absent. These are remaining requirements, rather than completed functionality.

7. Reproduction

From the repository root, change to the mission directory. Use a Python environment with NumPy for Parts 1 and 2; the notebook additionally imports Matplotlib. Parts 3 and 4 use the standard library.

```
Set-Location .\mission-05
python .\01_HNSW_Vector_Index.py --quick --out hnsw_local.json
python .\02_BM25_and_RRF_Combiner.py --quick
python .\02_BM25_and_RRF_Combiner.py --out hybrid_local.json
python .\03_NotebookLM_RAG_Audit.py --check
python .\04_Prompt_Injection_and_RAG_Audit_starter.py
```

For the briefing's 25,000-vector experiment, omit --quick and specify a separate output file to preserve saved evidence:

```
python .\01_HNSW_Vector_Index.py --n 25000 --out hnsw_25000_local.json
```

Part 2's --quick runs self-checks without the full benchmark. A fresh NotebookLM audit requires the external workspace, uploaded source documents, and manual inspection of the generated citations. Running the CSV aggregator alone does not repeat that experiment.

8. Completion status and interpretation

The repository demonstrates HNSW indexing, BM25/RRF retrieval components, a substantial recorded grounding audit, and an initial injection-scanning and overlap-scoring example. The local audit consistency check and safety demo passed. All mission Python files and the notebook/JSON structure were also checked for syntax or parsing errors during report preparation.

Remaining work includes reconciling the notebook and saved HNSW run, recording a reproducible full-hybrid benchmark, obtaining Document C, independently reviewing AI-assigned audit verdicts, completing the security and provenance requirements, and integrating Parts 2 through 4 into the notebook. Existing blank sign-off fields are not evidence of approval and have not been filled with invented signatures.

The contributor identity throughout this mission is Syed Hashir Sohail, Yasan Malik, Usman tariq qureshi, and Sufyan Khalid, group sustkever, Group 17. Research-paper authors remain credited as sources. Existing AI-assistance disclosures are retained to describe how audit judgments were produced.